DARWIX AI ENGINEER ASSESSMENT

Ayush Yadav

WHAT I BUILT

I built a local web app called Call Lab. It has a candidate-screening voice flow, a shared knowledge base, two localized finance demo flows, and live call alerts. The code, recordings, transcripts, tests, and setup steps are included.

I was given only the assessment PDF. No scripts, company policies, account data, or platform access were supplied. I therefore used fictional business rules and sample accounts. These are examples for testing, not real financial or employment information.

The speech recognition is real. The customer voices in the saved test calls are synthetic. I did not speak to real customers or claim native-speaker validation. The local web interface is the calling interface. No phone number is connected.

QUESTION 1 - GROUNDED VOICE AGENT

I chose candidate screening. A fictional internship asks whether the caller has built a Python project and how many hours they can spend each week. The approved demo rule is a Python project and at least 20 hours a week. Passing this check is only a preliminary match. A human makes the final decision.

The flow is: explain the demo, ask permission, ask about Python, ask about hours, then give a preliminary result. If hours are missing, the bot leaves them unconfirmed. If the caller changes an earlier number, the bot asks before replacing it. A stop request ends the flow. A request for a person creates a local mock escalation and clearly says that no person is connected.

FAQs, objections, policy answers, and qualification rules come from the Question 2 knowledge base. The bot returns approved source text and a citation. It does not have a large prompt containing all the answers. If it cannot find a supported answer, it says so. This makes the wording less flexible, but keeps the small demo predictable.

Three English recordings cover a cooperative caller, lack-of-experience objection, missing hours, changed hours, an unrelated question, and a request for a human. They are in evidence/q1_*.wav. Matching JSON files contain the reference speech, actual ASR output, source record, expected response, and result.

The optional business action is a mock screening summary, callback request, or escalation. These are kept in the local session and can be downloaded. No message or job application is sent to an outside service.

QUESTION 2 - KNOWLEDGE BASE

The input set has local text, saved HTML, a CSV table, and generated PDFs. It also has a duplicate page, an unapproved conflicting claim, a fake form with sample PII, a broken PDF, and a missing file. This lets the tests show what happens when input is messy.

For a website, I would first save an allowed page with its URL and retrieval time, then extract the main content. In this implementation I parse saved HTML. Navigation, headers, footers, scripts, and sidebars are removed. PDFs use text extraction. Scanned PDFs are flagged for OCR and review; OCR is not implemented. Tables keep their row values. Repeated lines and near-duplicate sections are removed. Failures are logged instead of silently producing empty records.

The manifest gives each section a stable topic, market, language, category, source file, owner, version, and ISO date. Dates are checked before indexing. English and local-language finance terms are listed as search keywords. Conflicting content in the sample is explicitly unapproved and excluded. The system does not automatically resolve every possible contradiction; a reviewer must approve policy changes.

Basic PII masking removes labelled names, email addresses, and phone or ID-like numbers from indexed content and saved conversation text. This is a small regex-based filter. It cannot find every unlabelled name, address, or sensitive medical detail. Only fictional data should be used in this prototype. Real use needs stronger PII detection and access controls.

The schema includes record_id, title, content, category, topic, market, language, source_ref, version, effective_date, owner, sha256, pii_redacted, and also_in. The hash identifies the source bytes. The also_in field keeps duplicate-source links. Qualification rules are stored as structured fields on the approved record.

Example: en_qualification-1, category qualification, market en, version 1.0, with a source link and minimum_hours set to 20. The sample records are in data/kb.json.

I keep short sections whole. Longer text is split at sentence boundaries around 160 words. I use BM25-style word scoring with a keyword boost, market filtering, and a minimum relevance check. There are no embeddings in this version. For this small approved set, lexical search is easy to inspect and test. For a larger set I would add multilingual embeddings, combine lexical and vector results, and test a reranker on labelled questions before using it.

The index currently contains 41 approved records. Retrieval passed 11/11 labelled queries. These queries were used during development, so they are not an independent benchmark. Each full result includes the retrieved text, source reference, expected topic, reason, timing, and verdict in evidence/retrieval_results.json.

- What work does the internship include? -> en_product-1; correct.
- What are the eligibility requirements? -> en_qualification-1; correct.
- What stipend do you pay? -> en_policy-1; correct.
- Is this remote or in the office? -> en_faq-1; correct.
- I am a fresher with no experience -> en_objection-1; correct.
- Mahal po, wala akong pambayad -> ph_objection-1; correct.
- What is a beneficiary or rider? -> ph_faq-1; correct.
- Berapa denda kalau terlambat? -> id_policy-1; correct.
- Saya belum bisa bayar -> id_objection-1; correct.
- Apa itu DP dan tenor? -> id_faq-1; correct.
- Who won the football match? -> safe fallback; correct.

QUESTION 3 - PHILIPPINES AND INDONESIA

The Philippines flow is a fictional life-insurance premium reminder. The default voice is fil-PH-BlessicaNeural. Recognition uses local faster-whisper base with the language set to tl. It accepts Filipino and Taglish. A caller can say English please to select English replies and English ASR, or Tagalog please to switch back. English support is tested in the text flow; pure-English speech in the Philippines flow has not been separately benchmarked.

Three localization choices for the Philippines:
1. The reminder uses polite po and asks permission before continuing. A budget objection gets an understanding reply instead of pressure to pay.
2. Premium, policy, beneficiary, rider, lapse, coverage, and bank referral stay in familiar insurance wording. The bot does not force an awkward literal translation.
3. The sample amount is spoken in pesos and the date is spoken out. Callback times use Philippine time. A late payment does not cause the bot to invent a lapse date or grace period.

The Indonesia flow is a fictional installment reminder. The default voice is id-ID-GadisNeural. Recognition uses the same local model but with language id. Replies stay in Indonesian. Colloquial words are accepted where the recognizer and keyword set support them.

Three localization choices for Indonesia:
1. The flow uses cicilan or angsuran for the payment, tenor for its term, DP for the down payment, denda for penalties, and jatuh tempo for the due date.
2. The sample amount is spoken as five hundred thousand rupiah. A callback must confirm WIB, WITA, or WIT rather than assume one timezone for the whole country.
3. A caller saying belum bisa bayar or a colloquial equivalent gets an offer of human help. The bot does not threaten the caller or promise a fee waiver, new tenor, or approval.

Each market has two synthetic call recordings with actual ASR transcripts. The Philippines tests include code-switching, an objection, a policy question, callback, and human help. Indonesia includes formal and colloquial speech, a fee question, finance terms, and human help.

I also ran a separate stress test using jv-ID-DimasNeural, a Javanese TTS voice, reading mixed Indonesian and Javanese wording. This is only a synthetic regional-language proxy. It is not proof of recognition quality for a real Javanese-accented Indonesian speaker. The results and audio are in evidence/regional_proxy.*. It misheard local words and sometimes used fallback. A real speaker from outside Jakarta still needs to test the system.

Measured call results:
- q1_cooperative: 3/3 expected responses; mean turn word error rate 25.0%.
- q1_objection: 4/4 expected responses; mean turn word error rate 12.5%.
- q1_conflict: 6/6 expected responses; mean turn word error rate 16.8%.
- q3_ph_cooperative: 3/4 expected responses; mean turn word error rate 58.9%.
- q3_ph_mixed: 4/4 expected responses; mean turn word error rate 31.2%.
- q3_id_cooperative: 4/4 expected responses; mean turn word error rate 11.2%.
- q3_id_colloquial: 3/4 expected responses; mean turn word error rate 26.7%.

Word error rate compares the recognized words with the synthetic script. The reported value is the mean of turn-level scores, not a population accuracy estimate. Numbers, spelling, and abbreviations can change that score even when meaning is preserved. Response counts compare the actual source selected with the expected topic.

Known gaps: a Filipino affordability sentence and an Indonesian regional phrase were misheard in the synthetic tests. The bot used fallback. The regional proxy also missed a human-help phrase. This is not good enough for a real launch. I would test a stronger speech model and a held-out set of native recordings, including phone audio, background noise, and code-switching. Local speakers and the business must review the scripts, consent wording, and policy terms.

QUESTION 4 - LIVE INSIGHTS

The browser can use a microphone or replay audio at normal speed. The replay sends four-second chunks only after that much audio time has elapsed. The backend transcribes each chunk and runs the alert rules immediately. The whole recording is not uploaded and analyzed after the call.

The agent/customer label comes from the selected audio channel. There is no automatic diarization. On a real call platform, separate media channels would provide this label. The prototype does not infer speaker identity from one mixed recording.

The rules detect a second-vehicle opportunity, a promise of guaranteed approval, a missing demo disclosure before asking for payment, frustration, payment difficulty, a callback request, and buying intent. Each alert suggests one short next step. For example, a second vehicle leads to asking about a separate quote, not inventing a multi-vehicle discount.

Controls include an ASR score cutoff of 0.50, negation checks, a short per-speaker context, a 30-second per-signal cooldown, priorities, topic labels, at most two alerts per chunk, and a 20-second display expiry. The ASR score is a heuristic, not a calibrated probability. I lowered the original cutoff after it missed clear synthetic phrases. That choice needs an independent human-audio evaluation.

Testing also found a false opportunity alert when a negative sentence was split between chunks. I changed the rule to require an ownership statement and added positive and negative chunk-boundary regression tests.

There is no LLM in the alert path. Its LLM time is therefore zero, not an omitted measurement. Timing starts when a complete audio chunk is available and continues through ASR, signal detection, response delivery, and client display. I also report time from the start of the chunk, which includes the four-second capture buffer.

Measured CLI replay latency:
- asr_ms: P50 662.6 ms; P95 1295.4 ms; 12 chunks.
- signal_ms: P50 0.0 ms; P95 0.5 ms; 12 chunks.
- llm_ms: P50 0.0 ms; P95 0.0 ms; 12 chunks.
- delivery_and_http_ms: P50 10.1 ms; P95 11.4 ms; 12 chunks.
- receive_to_display_ms: P50 673.3 ms; P95 1307.9 ms; 12 chunks.
- chunk_start_to_display_ms: P50 4670.7 ms; P95 5320.2 ms; 12 chunks.

These are local measurements on 12 chunks, not a production SLA. ASR time includes local model queueing

and cold start if present. Delivery is HTTP plus client overhead. Browser timing is saved separately in evidence/browser_latency.json. It measures client receipt and DOM update scheduling, not exact display paint time.

Observed live signals:
- q4_customer.wav: 2 chunks produced an alert before audio ended. Signals: second_vehicle, frustration, callback.
- q4_agent.wav: 1 chunks produced an alert before audio ended. Signals: risky_promise, missing_disclosure.
- q4_negative.wav: 0 chunks produced an alert before audio ended. Signals: none.
- q4_noise.wav: 0 chunks produced an alert before audio ended. Signals: none.

The text-only quality set has 13 cases. It recorded 0 false positives on 6 negative cases and 0 missed expected signals. The audio false-positive checks are listed in latency_report.json. They use one negative conversation and low-level synthetic noise. They do not establish performance on difficult real-world noise.

AT TEN TIMES THE LOAD

The current local speech model has one lock, so calls will queue. Session data is in memory and will be lost on restart. I would separate audio processing from the web server, use bounded queues, move session state into a shared store, and add worker capacity based on load tests. I would measure ASR delay and dropped chunks per call, not only total request counts.

For noisy audio, I would test better ASR, overlapping chunks, speech detection, and longer context. I would ask the caller to confirm dates, amounts, and unclear answers. Silence must not become a confident answer. A genuine regional-accent test and native-language review remain necessary.

WHAT IS INCLUDED AND WHAT IS LEFT

Included: working local source code, README, environment template, source fixtures, shared knowledge base, 42 automated tests with 0 failures, retrieval evidence, seven synthetic calls, transcripts, a regional proxy, live audio results, latency report, screenshot, and video walkthrough materials.

No API secrets or real customer data are included. The default setup needs no paid API account. First-time model download and online TTS need internet. The optional Groq path is implemented but not account-tested.

The prototype still needs approved business sources, real native-speaker calls, a stronger speech-quality check, authentication, encrypted storage where needed, tenant isolation, proper consent and retention rules, and a real human handoff before production use. The recordings are synthetic tests. Do not describe them as real customer recordings.

The video uses an automated screen recording and clearly labelled synthetic narration where included. A candidate should run the app, understand the code, and record a personal walkthrough before claiming they can explain every choice.

TECHNICAL REFERENCES

faster-whisper: https://github.com/SYSTRAN/faster-whisper
Edge TTS: https://github.com/rany2/edge-tts
Optional Groq speech API: https://console.groq.com/docs/speech-to-text

All financial and employment examples in this submission are fictional. These references explain tools, not business policy.